Procedural Generation

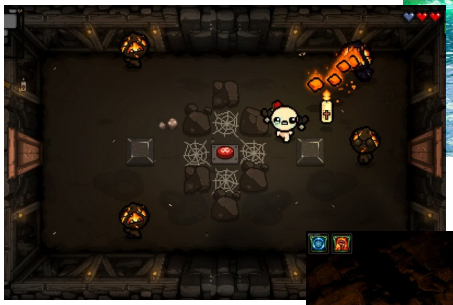
Will Jeynes
Sheffield Game Dev Soc

DISCUSS:
Example of a game with infinite worlds

To store every single minecraft world, would millions of times the storage space that exists in the entire world

Why procedural generation?

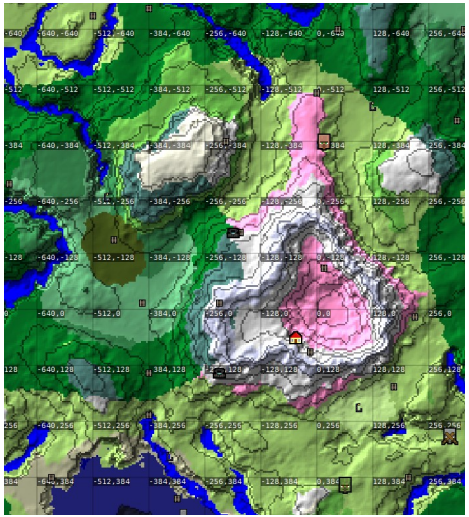
- Infinitley large worlds
- Replayability
- Time-Saving
- Dynamic Worlds
- Space Saving



Section 1: Preliminaries

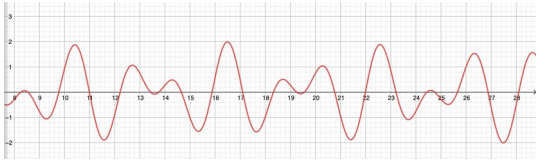
Seeded Random

- Computers suck at randomness
- Use complex mathematics and tricks for it to seem random
- We can use this to our advantage
  - By using a seed we can generate numbers that from the outside seem random BUT
  - Re-running the program with the same random requests will net the same results

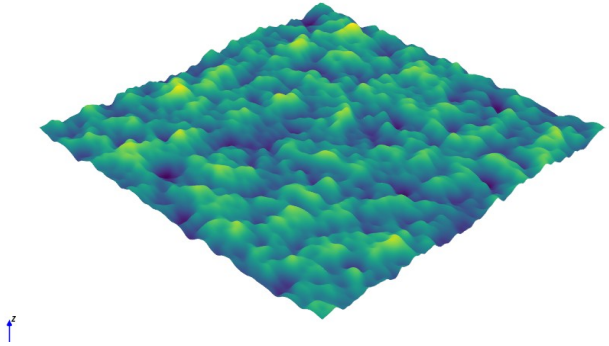


Perlin Noise

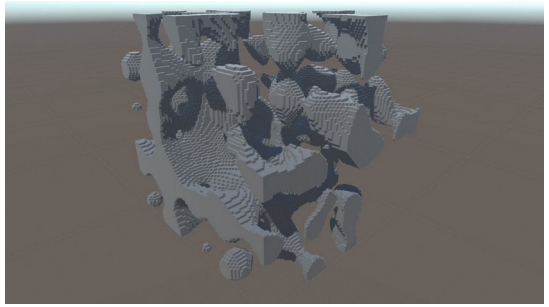
- Using completley random numbers for things like terrain height can look unnattural
- Perlin noise is a gradient of values 0-1, that move slowly over coordinate space
- Can make it random by offsetting any coorinates sampled



1D



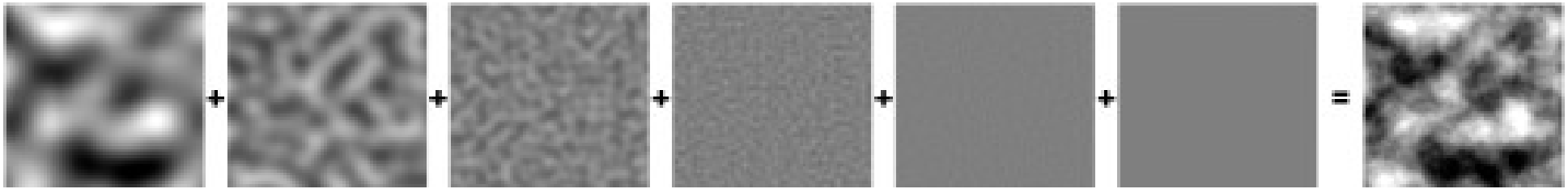
2D



3D

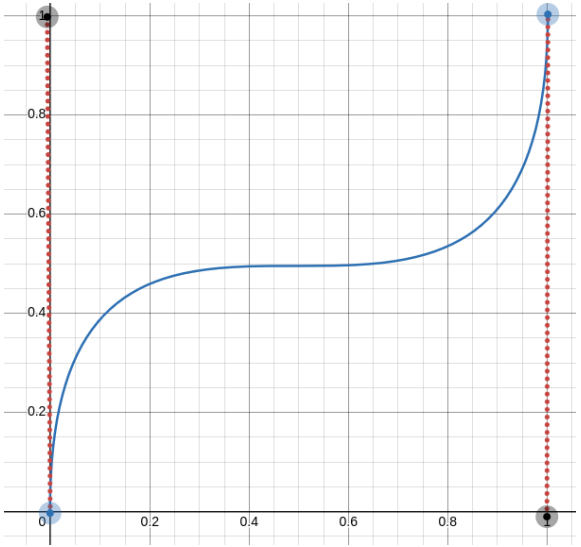
Perlin Noise - Octaves

- Perlin noise on its own can be a bit too smooth and unnatural
- Layer different noises together to add variation
- Multiply sample position to increase frequency of peaks and troughs
- Use lesser magnitudes to ensure not overpowered



Spline Points

- Having an even distribution of values might not be what we want
- We might want to favour the top and bottom, or the middle, or something more complicated
- One way to fix this problem:



Demo Details:

https://bit.ly/GenDemos

WASD to move around
Q to zoom out, R to zoom in
Random and Perlin generation should make sense

Section 2: Minecraft

How does minecraft do it?

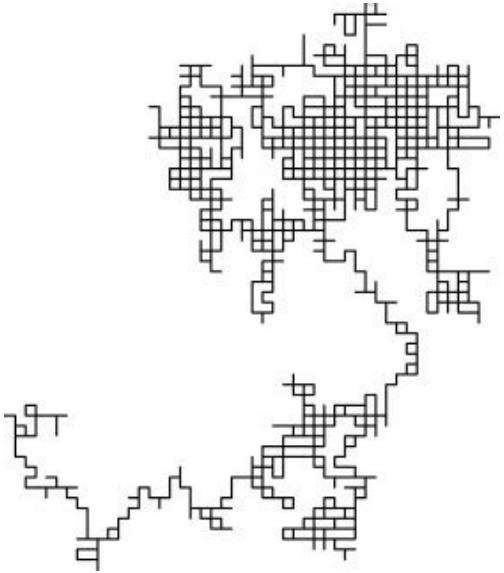
- All generation is seed based
- Uses 2D perlin noise for terrain height, 3D for caves
- Uses spline points to ensure cave and overworld generation look good
- Once world shape is in, surface vegetation and buildings are placed ontop

https://youtu.be/CSa5O6knuwI

Section 3: Other Generation Types

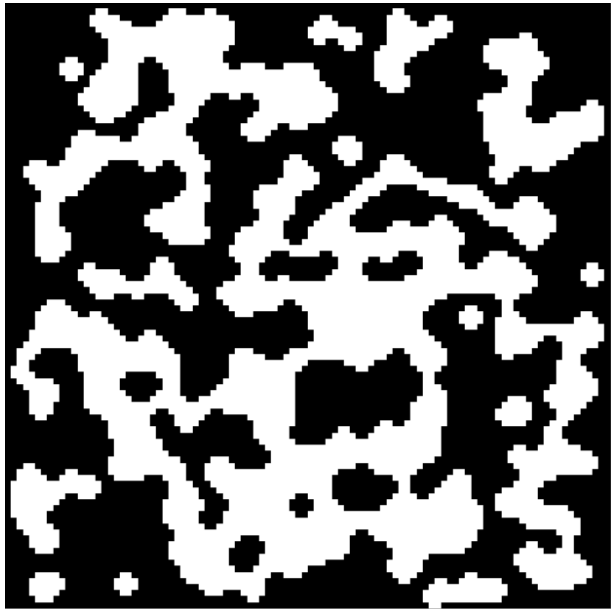
Random Walks

- Imagine a robot going around and placing items of the world
- He can be as smart or as dumb as needed
- Good for dungeons and connected rooms



Cellular Automata

- Another grid based generation method
- Evaluate a ruleset per tile at timestep
- Iterate over time for more interesting results



https://bzgeb.github.io/CellularAutomataGenerator/

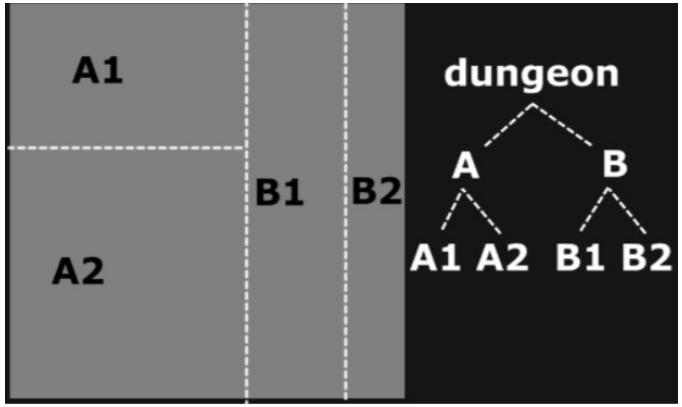
Grammars

- For when we want more predictable interactions
- Have a set of rules that must bew followed, generate around it
- Use in combination with other methods
- Still keeps replayability

```
Entrance > Room > Room > Room > Boss > Exit
```

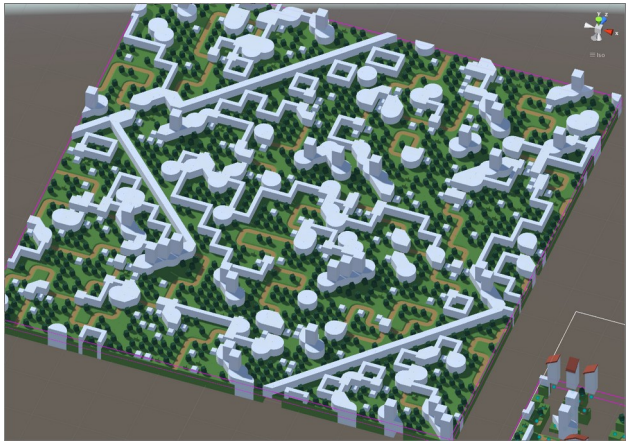
Binary Space Partitioning

- Split world down into smaller and smaller sizes
- Does not have to be even
- Players might see through grid based methods, this helps
- Many methods can be adapted for this paradigm



Wave Function Collapse

- Most implementations rely on the concept of “Wang Tiles”
  - Grid based building blocks
  - Rules as to what can be connected to each other (inside connected to inside e.t.c)
- Relies on the concept of Entropy
  - As tiles are placed, it is restricted what can and cannot be placed in a tile
  - Take advangate of tiles that need certain properties
- Running until failure givesa cohesive map
- Demo is a VERY VERY simple example, can be scaled up with more rules for some very cool stuff



Demo Details:

https://bit.ly/GenDemos

WASD to move around
Q to zoom out, R to zoom in
Radom Walk and WFC should make sense now

Thanks for listening